Task 3: Text-to-SQL Pipeline and Query Execution System

Goal

In Task 2, you manually broke natural language questions into structured components such as:

- Intent
- Tables
- Columns
- Filters
- Joins

In this task, you will use that structured understanding to build a simple Text-to-SQL system.

Your system should take the structured decomposition from Task 2 and transform it into executable SQL queries automatically.

The objective is to simulate the core workflow used in modern AI-powered database assistants.

How Task 2 Connects to Task 3

Task 2 focused on:

> Understanding the question before generating SQL.

Task 3 focuses on:

> Using that understanding to generate and execute SQL automatically.

You are now converting:

Natural Language
↓
Structured Understanding (Task 2)
↓
SQL Generation
↓
Query Execution
↓
Results

System You Need to Build

You will build a simple pipeline that performs the following steps:

Step 1: Input Question

Example:

"Show all orders placed by customers in Germany"

Step 2: Use Structured Decomposition

Reuse the decomposition format from Task 2.

Example:

Intent: Retrieve orders
Tables: orders, customers
Columns: orderNumber, customerName
Filters: country = 'Germany'
Joins: customers.customerNumber = orders.customerNumber

This decomposition can be:

- manually hardcoded,
- rule-based,
- or generated using an LLM prompt.

Step 3: Generate SQL Query

Your system must convert the structured decomposition into SQL.

Example:

```
SELECT o.orderNumber, c.customerName
FROM orders o
JOIN customers c
ON o.customerNumber = c.customerNumber
WHERE c.country = 'Germany';
```

Step 4: Execute SQL Query

Run the generated SQL query against PostgreSQL.

Step 5: Validate and Retry

If the query fails:

1. Read the database error message
2. Attempt to fix the query
3. Retry execution once

Example:

- wrong column name
- missing JOIN
- syntax issue

Step 6: Return Structured Output

Example Output:

```
{
 "question": "Show all orders placed by customers in Germany",
 "sql": "SELECT ...",
 "result": [],
 "status": "success"
}
```

Allowed Approaches

You may implement this system using either:

Option 1: Rule-Based Pipeline

Example:

- if question contains "count" → use COUNT(*)
- if multiple tables detected → add JOIN
- if filter exists → add WHERE clause

This approach uses predefined logic.

Option 2: Prompt Chaining with LLMs

You may use:

- OpenAI
- Claude

- Gemini
- or any LLM of your choice

Example flow:

Prompt 1 → Extract tables and columns
Prompt 2 → Generate SQL
Prompt 3 → Fix SQL errors if execution fails

This is called a prompt-chaining pipeline.

Important Rules

- Only SELECT queries are allowed
- DELETE / DROP / UPDATE / INSERT must be blocked
- Every SQL query execution must be logged
- The system must not crash on errors
- Maximum retry limit: 1 retry

Suggested Project Structure

```
project/
│
├── database.py
├── sql_generator.py
├── validator.py
├── executor.py
├── prompts/
├── logs/
└── main.py
```

What You Will Learn

By completing this task, you will learn:

- How Text-to-SQL systems work internally
- How structured reasoning improves SQL generation
- How LLM pipelines are designed
- How to validate and execute SQL safely
- How AI systems recover from execution errors

Important Thinking Question

While building your system, think about:

> “How does the system know whether the generated SQL is actually correct?”

This question will directly connect to the next task:

- evaluation,
- benchmarking,
- and agentic self-correction systems.

Deliverables

Please submit the following:

- Source code for your Text-to-SQL pipeline
- Generated SQL outputs
- Query execution logs
- Short explanation of your pipeline architecture and design decisions
- Example successful query cases
- Example failed query cases and retry handling behavior

Evaluation Requirement (IMPORTANT)

Your system must be evaluated using the benchmark question dataset provided below:

Text-to-SQL Benchmark Dataset

You are required to:

- Run your system against the test questions in the dataset
- Generate SQL queries automatically
- Execute the generated SQL queries
- Compare outputs against expected behavior or manually verified SQL results

---

Suggested Evaluation Metrics

You should evaluate your system using metrics such as:

- SQL execution success rate
- Correctness of generated SQL
- Correct table and column selection
- Query result accuracy
- Retry/self-correction success rate
- Number of failed queries
- Query generation latency
- Natural language response quality

Expected Evaluation Output

You are encouraged to prepare an evaluation report or table similar to the following:

Question	Generated SQL	Executed Successfully	Correct Result	Retry Needed	Final Status
Count customers in USA	Yes	Yes	Yes	No	Success
Orders from Germany	Yes	No	Fixed After Retry	Yes	Success